RegRadar Feature Ticket List

Contents

RegRadar — Feature Ticket List

Version 1.0 | Buildable tickets derived from the PRD, Technical Architecture, Security & Access, and Frontend Specification documents

---

How to Use This Document

Every ticket below follows the same shape: what to build, how you'll know it's actually done, what has to exist first, and how urgent it is. The last field in every ticket — **AI Coding Prompt** — is written to be copy-pasted directly into an AI coding tool (Claude Code, Cursor, etc.) as a self-contained task description. It repeats the relevant tech-stack and file-path context on purpose, so it works even if you paste it in on its own, in a fresh session, without the rest of this document attached.

Ticket IDs are grouped by epic prefix (`FOUND`, `ING`, `AGENT`, `API`, `SEC`, `EVAL`, `DELIV`, `FE`, `LAUNCH`) so dependencies are easy to trace at a glance.

Priority labels mean exactly this: **Must-have** — the product does not function as a V1 launch without it. **Should-have** — the product works without it, but it's a real gap a customer would notice quickly; build it in the first couple of iterations after launch if not before. **Nice-to-have** — explicitly deferred scope (V2+ per the PRD's non-goals), included here so it's planned, not forgotten.

Dependencies list other ticket IDs that must be functionally complete first. A ticket with no dependencies can start immediately.

Ticket Index

ID	Feature	Priority	Depends On
FOUND-01	Repository scaffolding	Must-have	None
FOUND-02	Database schema & migrations	Must-have	FOUND-01
FOUND-03	Local dev environment (Docker Compose)	Must-have	FOUND-02
FOUND-04	CI pipeline	Must-have	FOUND-01
FOUND-05	Configuration & secrets management	Must-have	FOUND-01
ING-01	SEC EDGAR ingestion connector	Must-have	FOUND-02, FOUND-05

ID	Feature	Priority	Depends On
ING-02	FDA RSS ingestion connector	Must-have	FOUND-02, FOUND-05
ING-03	FINRA feed ingestion connector	Should-have	FOUND-02, FOUND-05
ING-04	Prefect scheduling orchestration	Must-have	ING-01, ING-02
ING-05	Raw PDF storage to S3 with dedup	Must-have	ING-01
ING-06	Redis/Celery job queue wiring	Must-have	FOUND-03
AGENT-01	LangGraph supervisor graph & typed state	Must-have	FOUND-02, ING-06
AGENT-02	Triage Agent (zero-shot classification)	Must-have	AGENT-01
AGENT-03	Dual-model voting for low-confidence classification	Should-have	AGENT-02
AGENT-04	PDF chunking pipeline	Must-have	ING-05
AGENT-05	Embedding & pgvector indexing	Must-have	AGENT-04
AGENT-06	RAG retrieval agent (hybrid search + rerank)	Must-have	AGENT-05
AGENT-07	Analysis Agent (structured extraction)	Must-have	AGENT-06
AGENT-08	Summarization Agent (persona briefs)	Must-have	AGENT-07
AGENT-09	Tiered model routing	Should-have	AGENT-02, AGENT-08
AGENT-10	Delivery Agent	Must-have	AGENT-08
API-01	FastAPI skeleton + health check	Must-have	FOUND-02
API-02	API key authentication	Must-have	API-01, SEC-01
API-03	Rate limiting middleware	Must-have	API-02
API-04	GET /v1/filings	Must-have	API-02, AGENT-01
API-05	GET /v1/filings/{id}	Must-have	API-04
API-06	POST /v1/filings/search (RAG Q&A)	Must-have	AGENT-06, API-02
API-07	GET /v1/filings/{id}/brief	Must-have	AGENT-08, API-05
API-08	Webhook registration & HMAC signing	Must-have	API-02, SEC-01
API-09	GET /v1/metrics	Should-have	EVAL-01, EVAL-06
API-10	POST /v1/config/sources	Should-have	API-02
API-11	GET /v1/me (session/role resolution)	Nice-to-have	API-02, FE-02
SEC-01	Row-level security policies	Must-have	FOUND-02
SEC-02	Webhook URL SSRF validation	Must-have	API-08

ID	Feature	Priority	Depends On
SEC-03	Webhook replay protection	Must-have	API-08
SEC-04	Duplicate filing race protection	Must-have	ING-01
SEC-05	Organization-scoping scaffolding	Should-have	SEC-01
EVAL-01	Ragas faithfulness + context recall harness	Must-have	AGENT-06, AGENT-07
EVAL-02	ROUGE-L summarization regression suite	Must-have	AGENT-08
EVAL-03	Alert precision/recall test set	Should-have	AGENT-02
EVAL-04	Extraction F1 eval	Should-have	AGENT-07
EVAL-05	LangSmith prompt versioning integration	Must-have	AGENT-01
EVAL-06	Langfuse cost/latency dashboard & budget breaker	Must-have	AGENT-09
EVAL-07	CI eval gate	Must-have	EVAL-01, EVAL-02, FOUND-04
DELIV-01	Slack delivery integration	Must-have	AGENT-10
DELIV-02	SendGrid email integration	Must-have	AGENT-10
DELIV-03	Weekly digest job	Should-have	DELIV-01, DELIV-02
DELIV-04	Delivery failure fallback	Should-have	DELIV-01, DELIV-02
FE-01	Design system / Tailwind token implementation	Nice-to-have	None
FE-02	SSO login & session integration	Nice-to-have	API-02
FE-03	Filings List screen	Nice-to-have	FE-01, FE-02, API-04
FE-04	Filing Detail screen	Nice-to-have	FE-03, API-05, API-07
FE-05	Ask RegRadar (RAG search) screen	Nice-to-have	FE-01, API-06
FE-06	Webhooks management screen	Nice-to-have	FE-01, API-08
FE-07	API Keys management screen	Nice-to-have	FE-01, FE-02
FE-08	Metrics & Cost dashboard screen	Nice-to-have	FE-01, API-09
FE-09	Source Configuration screen	Nice-to-have	FE-01, API-10
LAUNCH-01	Load testing (100 concurrent filings)	Must-have	API-04 through API-08, DELIV-01, DELIV-02
LAUNCH-02	Known-limitations documentation	Should-have	EVAL-01 through EVAL-04
LAUNCH-03	OCR fallback for scanned PDFs	Nice-to-have	AGENT-04
LAUNCH-04	Architecture Decision Records for build-time choices	Should-have	None (ongoing)

Epic: Foundation

FOUND-01 — Repository Scaffolding

Priority: Must-have · **Depends on:** None

What to build: The full monorepo layout from the Technical Architecture Document — `src/regradar/` with `agents/`, `ingestion/`, `rag/`, `models/`, `schemas/`, `api/`, `workers/`, `delivery/`, `eval/`, `llm_routing/`, `observability/`, and `core/` subpackages, plus `migrations/`, `tests/`, `docs/adrs/`, and `infra/docker/`. Every package gets an `__init__.py` and a one-line docstring describing its purpose; no business logic yet.

Acceptance criteria: - [ ] Directory structure matches the Technical Architecture Document's file tree exactly - [ ] `pyproject.toml` defines the project with Python 3.11+, and dependency groups for `api`, `worker`, dev - [ ] A fresh clone can run `pip install -e .` without errors - [ ] `README.md` explains how to run the project locally in under 10 steps

AI Coding Prompt: > Scaffold a Python monorepo for a project called RegRadar using this exact structure: `src/regradar/{agents,ingestion,rag,models,schemas,api,workers,delivery,eval,llm_routing,c` `migrations/`, `tests/{unit,integration,fixtures}/`, `docs/adrs/`, `infra/docker/`. Use `pyproject.toml` (not `setup.py`) targeting Python 3.11+, with FastAPI, SQLAlchemy 2.x, Pydantic v2, Celery, and LangGraph as core dependencies. Add an `__init__.py` with a one-sentence docstring to every package explaining its responsibility. Do not implement any logic yet — this is scaffolding only. Include a README with local setup instructions.

FOUND-02 — Database Schema & Migrations

Priority: Must-have · **Depends on:** FOUND-01

What to build: All nine tables from the Technical Architecture Document's schema (`filings`, `filing_chunks`, `extractions`, `briefs`, `webhooks`, `deliveries`, `source_configs`, `eval_runs`, `api_keys`) as SQLAlchemy ORM models in `src/regradar/models/`, with Alembic migrations that create them in order, respecting foreign keys. Enable the `pgvector` Postgres extension as the very first migration.

Acceptance criteria: - [ ] `CREATE EXTENSION IF NOT EXISTS vector;` runs as migration 0001, before any table with a `VECTOR` column - [ ] All nine tables exist with the exact fields, types, and relationships specified in the Technical Architecture Document - [ ] `filing_chunks.embedding` is a working `VECTOR(1536)` column - [ ] `alembic upgrade head` and `alembic downgrade base` both run cleanly with no errors - [ ] Every foreign key relationship (filings → chunks/extractions/briefs/deliveries, webhooks → deliveries, api_keys → webhooks) is enforced at the database level

AI Coding Prompt: > In the `src/regradar/models/` package, create SQLAlchemy 2.x ORM models for these nine tables: `filings` (id, source enum[SEC/FDA/FINRA], source_document_id, entity_name, filing_type, filing_url, raw_pdf_s3_key, published_at, ingested_at, status enum, domain enum[financial/clinical/environmental/other], risk_level enum[low/medium/high/critical], priority_score float, classification_confidence float, processing_error, created_at, updated_at); `filing_chunks` (id, filing_id FK, chunk_index, chunk_text, section_reference, token_count, embedding VECTOR(1536), is_table bool, created_at); `extractions` (id, filing_id FK unique, obligations JSONB, deadlines JSONB, risk_flags JSONB, affected_products JSONB, key_entities JSONB, competitor_mentions JSONB, model_used, raw_model_response JSONB, created_at); `briefs` (id, filing_id FK unique, executive_brief, cco_summary, analyst_summary, engineer_summary, model_used, rouge_l_score, created_at); webhooks (id, api_key_id FK, url, hmac_secret, is_active, filter_domain, filter_min_risk, created_at); `deliveries` (id, filing_id FK, channel enum[slack/email/webhook], webhook_id FK nullable, recipient, status enum, response_code, attempt_count, sent_at, created_at); `source_configs` (id, source enum, domains array, is_active, poll_interval_seconds, last_polled_at,

last_etag, updated_at); eval_runs (id, run_type enum, prompt_version, git_commit_sha, ragas_faithfulness, ragas_context_recall, rouge_l, alert_precision, alert_recall, hallucination_rate, extraction_f1, p99_latency_ms, avg_cost_per_filing_usd, passed bool, created_at); api_keys (id, key_hash, owner_label, is_active, rate_limit_per_minute, created_at, last_used_at). Set up Alembic migrations under `migrations/`, with the first migration enabling the `pgvector` Postgres extension before any other table is created.

FOUND-03 — Local Dev Environment (Docker Compose)

Priority: Must-have · **Depends on:** FOUND-02

What to build: A `docker-compose.yml` under `infra/` that starts Postgres (with pgvector), Redis, the FastAPI app, and one Celery worker, all wired together with sane defaults, so `docker compose up` gets a working local stack with zero external API calls required.

Acceptance criteria: - [ ] `docker compose up` starts all four services successfully on a clean machine - [ ] The API's health check endpoint responds `200 OK` within 30 seconds of startup - [ ] Migrations run automatically on Postgres container startup (or via a documented one-line command) - [ ] The stack works entirely offline except for calls this specific service intentionally makes outward (none should be required just to boot)

AI Coding Prompt: > Write a `docker-compose.yml` at `infra/docker-compose.yml` for a Python project called RegRadar. It needs four services: `postgres` (use the `pgvector/pgvector:pg16` image, expose 5432, with a healthcheck), `redis` (redis:7-alpine, expose 6379), `api` (build from `infra/docker/Dockerfile.api`, depends on postgres and redis being healthy, expose 8000, run `uvicorn regradar.api.main:app`), and `worker` (build from `infra/docker/Dockerfile.worker`, depends on postgres and redis, run a Celery worker against `regradar.workers.celery_app`). Also write both Dockerfiles using a `python:3.11-slim` base. Use environment variables (not hardcoded values) for all connection strings, sourced from a `.env` file at the repo root.

FOUND-04 — CI Pipeline

Priority: Must-have · **Depends on:** FOUND-01

What to build: A GitHub Actions workflow that runs on every pull request: linting, type-checking, and unit tests. This is separate from the eval-gate workflow (EVAL-07), which runs the AI-quality regression suite specifically.

Acceptance criteria: - [ ] `.github/workflows/ci.yml` runs `ruff` (or equivalent linter), `mypy` (or equivalent type-checker), and `pytest` on every PR - [ ] A PR with a failing test or lint error is blocked from merging (branch protection documented, even if not configurable by the AI tool itself) - [ ] The workflow completes in under 5 minutes on a typical PR

AI Coding Prompt: > Create `.github/workflows/ci.yml` for a Python 3.11 project using `pyproject.toml`. On every pull request, it should: check out the repo, set up Python 3.11 with pip caching, install the project with `pip install -e ".[dev]"`, run `ruff check .`, run `mypy src/`, and run `pytest tests/unit -v`. Fail the workflow if any step fails. Keep it under 5 minutes by caching dependencies between runs.

FOUND-05 — Configuration & Secrets Management

Priority: Must-have · **Depends on:** FOUND-01

What to build: A single `core/config.py` using Pydantic Settings that loads every environment variable listed in the Technical Architecture Document's `.env` reference (database, Redis, S3, OpenAI,

Hugging Face, ingestion sources, Prefect, delivery, eval/observability, API, and deployment settings) into one typed, validated settings object used everywhere else in the codebase.

Acceptance criteria: - [ ] All environment variables from the Technical Architecture Document's `.env` reference are represented as typed fields - [ ] The app fails fast at startup with a clear error if a required variable is missing, rather than failing later with a confusing error deep in the code - [ ] Secrets are never logged, even at debug log level - [ ] `.env.example` exists at the repo root with every variable name and a blank or placeholder value, and is the only `.env*` file committed to version control

AI Coding Prompt: > In `src/regradar/core/config.py`, create a Pydantic Settings (`pydantic-settings`) class named `Settings` that loads these environment variables with correct types and sensible defaults where noted: `ENV` (str, default "development"), `APP_SECRET_KEY` (str, required), `DATABASE_URL` (str, required), `REDIS_URL` (str, required), `S3_BUCKET_NAME`, `S3_REGION`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` (all required), `OPENAI_API_KEY`, `HUGGINGFACE_API_TOKEN` (required), `TIER_HIGH_MODEL` (default "gpt-4o"), `TIER_LOW_MODEL` (default "granite-13b"), `SEC_EDGAR_USER_AGEN` (required, must include a contact email), `SEC_EDGAR_RATE_LIMIT_PER_SEC` (int, default 10), `INGESTION_POLL_INTERVAL_SECONDS` (int, default 300), `SLACK_WEBHOOK_URL`, `SENDGRID_API_KEY`, `LANGSMITH_API_KEY`, `LANGFUSE_PUBLIC_KEY`, `LANGFUSE_SECRET_KEY`, `DAILY_COST_ALERT_THRESHOLD_USD` (float, default 50), `API_RATE_LIMIT_PER_MINUTE_DEFAULT` (int, default 60). Raise a clear validation error at import time if any required field is missing. Add a `.env.example` at the repo root listing every one of these variable names with empty or placeholder values, and add `.env` to `.gitignore`.

Epic: Ingestion

ING-01 — SEC EDGAR Ingestion Connector

Priority: Must-have · **Depends on:** FOUND-02, FOUND-05

What to build: A connector in `src/regradar/ingestion/sources/sec_edgar.py` that polls SEC EDGAR's submissions and full-text search APIs, detects new filings since the last check, and writes new `filings` rows with `status = ingested`.

Acceptance criteria: - [ ] Every outbound request includes the required `User-Agent` header from `SEC_EDGAR_USER_AGENT` - [ ] Requests respect the 10 req/sec rate limit with client-side throttling - [ ] New filings are detected using ETag/Last-Modified comparison against `source_configs.last_etag`, not by re-downloading and diffing full content - [ ] A filing already present (matched by `source_document_id`) is never inserted twice - [ ] Unit tests cover: a normal new-filing case, a rate-limit response (429), and an EDGAR outage (connection error) — all handled without crashing the ingestion process

AI Coding Prompt: > Implement `src/regradar/ingestion/sources/sec_edgar.py`. Write a function `poll_edgar(source_config: SourceConfig) -> list[NewFiling]` that calls the SEC EDGAR submissions API (`https://data.sec.gov/submissions/CIK{cik}.json`) and full-text search API (`https://efts.sec.gov/LATEST/search-index`), using the `SEC_EDGAR_USER_AGENT` setting as the `User-Agent` header on every request (EDGAR rejects requests without a descriptive one). Throttle requests to respect `SEC_EDGAR_RATE_LIMIT_PER_SEC` using a token-bucket or simple sleep-based limiter. Use the ETag/`Last-Modified` response headers, compared against `source_configs.last_etag`, to detect whether there's new content before parsing anything. For each genuinely new filing, check `source_document_id` doesn't already exist in the `filings` table before inserting (use a database-level unique constraint on `source_document_id`, not just an application-level check, to prevent race conditions). Write unit tests with mocked HTTP responses covering: a successful new-filing response, a 429 rate-limit response, and a connection timeout — all three should be handled gracefully without raising an unhandled exception.

ING-02 — FDA RSS Ingestion Connector

Priority: Must-have · **Depends on:** FOUND-02, FOUND-05

What to build: A connector in `src/regradar/ingestion/sources/fda_rss.py` that polls the configured FDA RSS feed URL and inserts new `filings` rows for items not seen before.

Acceptance criteria: - [ ] Feed URL is read from `source_configs`, not hardcoded - [ ] New items are detected by comparing feed entry GUIDs/links against previously seen `source_document_id` values - [ ] Malformed or partially broken XML doesn't crash the poll — it logs a warning and continues with whatever entries did parse - [ ] Unit tests cover a normal feed, an empty feed, and a malformed-XML feed

AI Coding Prompt: > Implement `src/regradar/ingestion/sources/fda_rss.py`. Write a function `poll_fda_rss(source_config: SourceConfig) -> list[NewFiling]` that fetches the RSS feed URL from `source_config` (using the `feedparser` library), extracts each `<item>`'s GUID, title, link, and publish date, and returns only entries whose GUID isn't already present as a `source_document_id` in the `filings` table. Wrap the XML parsing so a malformed feed logs a warning and returns whatever entries parsed successfully rather than raising an exception that would block the whole ingestion run. Write unit tests using `tests/fixtures/sample_filings/` style fixture feed files for a normal feed, an empty feed, and a malformed feed.

ING-03 — FINRA Feed Ingestion Connector

Priority: Should-have · **Depends on:** FOUND-02, FOUND-05

What to build: The FINRA equivalent of ING-01/ING-02, following whatever public interface FINRA exposes for rulemaking notices (confirm the exact feed/API shape during implementation, since it's less standardized than EDGAR).

Acceptance criteria: - [ ] New FINRA notices are detected and inserted as `filings` rows with `source = FINRA` - [ ] Same dedup guarantee as ING-01/ING-02 (no duplicate `source_document_id`) - [ ] Failure behavior matches the other two connectors (log and continue, never crash the scheduler)

AI Coding Prompt: > Implement `src/regradar/ingestion/sources/finra_feed.py`, following the same pattern as `sec_edgar.py` and `fda_rss.py` in this codebase: a `poll_finra(source_config: SourceConfig) -> list[NewFiling]` function that detects new FINRA rulemaking notices and regulatory notices, dedups against existing `source_document_id` values in the `filings` table, and fails gracefully (log + continue) on any request or parsing error. Research FINRA's current public feed or API for rulemaking notices before implementing, since it does not have the same standardized JSON API as SEC EDGAR — document whatever endpoint is used in a comment at the top of the file.

ING-04 — Prefect Scheduling Orchestration

Priority: Must-have · **Depends on:** ING-01, ING-02

What to build: A Prefect flow in `src/regradar/ingestion/flows.py` that runs all active source connectors on a schedule (every `INGESTION_POLL_INTERVAL_SECONDS`), with retry and backoff on failure.

Acceptance criteria: - [ ] The flow runs each active `source_configs` row's connector and updates `last_polled_at` and `last_etag` after each run - [ ] A failed connector run retries with exponential backoff up to a fixed maximum before giving up for that cycle - [ ] One source failing (e.g., FDA down) never prevents the others (e.g., SEC EDGAR) from running in the same cycle - [ ] Prefect UI shows run history, duration, and failure reasons for every scheduled run

AI Coding Prompt: > In `src/regradar/ingestion/flows.py`, write a Prefect 2.x flow named `poll_all_sources` that queries the `source_configs` table for all rows where `is_active = true`,

and for each one, calls the matching connector (`poll_edgar`, `poll_fda_rss`, or `poll_finra` depending on the `source` field) as a Prefect task with `retries=3` and exponential backoff. Each task failure should be caught and logged without stopping the other tasks in the same flow run (use `allow_failure` or equivalent so one source's outage doesn't block the others). After each successful task, update that source's `last_polled_at` and `last_etag` fields. Schedule this flow to run every `INGESTION_POLL_INTERVAL_SECONDS` seconds using a Prefect deployment with an interval schedule.

ING-05 — Raw PDF Storage to S3 with Dedup

Priority: Must-have · **Depends on:** ING-01

What to build: A shared `pdf_intake.py` module used by every ingestion connector to download a filing's PDF, hash it, upload it to S3, and store the resulting key on the `filings` row.

Acceptance criteria: - [ ] Downloaded PDFs are hashed (e.g., SHA-256) before upload, and an identical PDF already in S3 is not re-uploaded - [ ] `filings.raw_pdf_s3_key` is set only after a successful upload — never left pointing at a non-existent object - [ ] A failed download is retried once, then the filing is marked `status = failed` with `processing_error` populated, rather than left in limbo - [ ] Unit tests cover a successful upload, a duplicate-content upload (should skip re-upload), and a failed download

AI Coding Prompt: > In `src/regradar/ingestion/pdf_intake.py`, write a function `intake_pdf(url: str, filing_id: UUID) -> str` that downloads a PDF from `url`, computes its SHA-256 hash, checks whether an object with that hash already exists in the configured S3 bucket (use the hash as part of the S3 key path to make this check cheap), uploads it only if it doesn't already exist, and returns the S3 key. On download failure, retry once with a short delay; if it still fails, update the corresponding `filings` row to `status = "failed"` with a descriptive `processing_error`, and raise a custom `PdfIntakeError` so the calling ingestion flow can decide how to proceed. Write unit tests with a mocked S3 client (use `moto` or an equivalent) covering: successful upload, duplicate-hash skip, and download failure.

ING-06 — Redis/Celery Job Queue Wiring

Priority: Must-have · **Depends on:** FOUND-03

What to build: The Celery app configuration (`workers/celery_app.py`) that connects to Redis as both broker and result backend, and a `pipeline_tasks.py` task that the Ingestion Agent enqueues once a filing is stored, to hand off to the agent pipeline asynchronously.

Acceptance criteria: - [ ] `celery -A regradar.workers.celery_app worker` starts cleanly against the local Redis instance - [ ] Enqueuing a task for a `filing_id` is a single function call from the ingestion code, with no direct Celery API leakage into `ingestion/` - [ ] A task failure doesn't silently disappear — it's retried per Celery's retry policy and, after exhausting retries, the filing is marked `status = failed`

AI Coding Prompt: > In `src/regradar/workers/celery_app.py`, configure a Celery app using REDIS_URL for both broker and result backend. In `src/regradar/workers/pipeline_tasks.py`, define a task `process_filing(filing_id: str)` with `max_retries=3` and exponential backoff, decorated appropriately for Celery, that will eventually invoke the LangGraph pipeline (stub this call for now — the real graph is built in AGENT-01). Provide a simple `enqueue_filing_processing(filing_id: UUID)` helper function in the same module that ingestion code can call without importing Celery internals directly. On final retry exhaustion, update the `filings` row's `status` to `"failed"`.

Epic: Multi-Agent Pipeline

AGENT-01 — LangGraph Supervisor Graph & Typed State

Priority: Must-have · **Depends on:** FOUND-02, ING-06

What to build: The core LangGraph graph definition in `agents/graph.py` and the shared Pydantic state schema in `agents/state.py` that every subsequent agent reads from and writes to. This is the backbone every other AGENT ticket plugs into.

Acceptance criteria: - [ ] `state.py` defines a single Pydantic model representing everything that flows through the pipeline: filing metadata, classification result, retrieved context, extraction result, brief text, delivery status - [ ] `graph.py` wires together placeholder nodes for all six agents (ingestion is external to the graph; triage, retrieval, analysis, summarization, delivery are graph nodes) with conditional edges — specifically, a low-priority filing can skip the deep RAG retrieval step - [ ] The graph can run end-to-end against a fixture filing with stub agent implementations and produce a completed state object - [ ] Each node can be tested in isolation by constructing a state object and calling the node function directly, without running the whole graph

AI Coding Prompt: > In `src/regradar/agents/state.py`, define a Pydantic model `PipelineState` with fields for: `filing_id`, `raw_text`, `domain` (optional, set by triage), `risk_level` (optional), `classification_confidence` (optional), `retrieved_chunks` (optional list), `extraction` (optional structured object with obligations/deadlines/risk_flags/affected_products/key_entities), `briefs` (optional object with executive_brief/cco_summary/analyst_summary/engineer_summary), and `delivery_status` (optional). In `src/regradar/agents/graph.py`, build a LangGraph `StateGraph` using `PipelineState` with nodes for `triage`, `retrieve`, `analyze`, `summarize`, and `deliver` (import them from their respective agent modules — stub each with a function that just returns the state unchanged for now). Add a conditional edge after `triage`: if `risk_level` is "low", skip the `retrieve` node and go straight to `analyze`; otherwise run `retrieve` first. Write an integration test that runs the full graph against a fixture `PipelineState` and asserts it completes without error and reaches the `deliver` node.

AGENT-02 — Triage Agent (Zero-Shot Classification)

Priority: Must-have · **Depends on:** AGENT-01

What to build: The real implementation of the `triage` node — calls Hugging Face's zero-shot classification model to assign `domain` and an initial `risk_level`/`priority_score`.

Acceptance criteria: - [ ] Calls `facebook/bart-large-mnli` via the HF Inference API with candidate labels `financial`, `clinical`, `environmental`, `other` - [ ] Writes `domain`, `classification_confidence`, and an initial `risk_level` to the state and to the `filings` row - [ ] Achieves >90% accuracy against a manually labeled 100-filing test set (per PRD goal G2) — a test asserting this against the fixture set is part of the deliverable - [ ] On HF API failure, retries once, then leaves the filing in a `needs_classification` status rather than guessing

AI Coding Prompt: > Implement `src/regradar/agents/triage_agent.py` as the real `triage` node function for the LangGraph pipeline defined in `agents/graph.py`. It should call the Hugging Face Inference API's zero-shot classification endpoint for `facebook/bart-large-mnli` (POST `https://api-inference.huggingface.co/models/facebook/bart-large-mnli`) with `candidate_labels: ["financial", "clinical", "environmental", "other"]` against the filing's extracted text. Take the top-scoring label as `domain` and the score as `classification_confidence`. Derive an initial `risk_level` using a simple keyword-rule + confidence heuristic (document the rule set in a docstring — this will be refined later). Update both the `PipelineState` and the corresponding `filings` database row. On an HTTP error or timeout from the HF API, retry once after a short delay; if it still fails, set `filings.status = "needs_classification"` instead of guessing a risk level. Write a test using the 100-filing labeled fixture set described in `tests/fixtures/` and assert overall classification accuracy exceeds 90%.

AGENT-03 — Dual-Model Voting for Low-Confidence Classification

Priority: Should-have · **Depends on:** AGENT-02

What to build: A safety net for PRD risk R5 (misclassification cascade) — when the zero-shot classifier's confidence is below a configurable threshold, run a second GPT-4o-based spot check and use the more cautious (higher-risk) of the two results.

Acceptance criteria: - [ ] A confidence threshold is a configuration value, not hardcoded - [ ] Below the threshold, a second classification call is made using GPT-4o with a distinct prompt - [ ] When the two results disagree, the higher-risk result wins, and both results are logged for later analysis - [ ] This path is only triggered below the threshold — normal-confidence classifications don't incur the extra cost

AI Coding Prompt: > Extend `src/regradar/agents/triage_agent.py` (built in AGENT-02) with a dual-model voting safety net. Add a configuration value `CLASSIFICATION_CONFIDENCE_THRESHOLD` (default 0.75). If the zero-shot classifier's confidence is below this threshold, make a second classification call using GPT-4o with a prompt asking it to classify the same filing into the same four domain categories and assign a risk level with reasoning. If the two results disagree on risk level, use whichever is higher-severity (Critical > High > Medium > Low). Log both results (model name, domain, risk_level, confidence/reasoning) to a structured log or the `raw_model_response` equivalent for later analysis, regardless of whether they agreed. Write a test confirming the second model is only called when confidence is below the threshold, and that disagreement resolves to the higher-severity result.

AGENT-04 — PDF Chunking Pipeline

Priority: Must-have · **Depends on:** ING-05

What to build: `rag/chunking.py` — splits a filing's extracted text into 512-token chunks with 50-token overlap, aware of section boundaries, and flags table-derived chunks.

Acceptance criteria: - [ ] Chunk size and overlap are configuration values, defaulting to 512/50 tokens - [ ] Chunks respect section boundaries where detectable (e.g., don't split mid-sentence at a hard 512-token cutoff if a natural section break is nearby) - [ ] Chunks derived from detected tables are flagged (`is_table = true`) - [ ] A test using at least one fixture from `tests/fixtures/sample_filings/` (including a table-heavy PDF) verifies chunk boundaries and table-flagging behave as expected

AI Coding Prompt: > Implement `src/regradar/rag/chunking.py` with a function `chunk_filing(text: str, tables: list[TableBlock]) -> list[Chunk]` that splits `text` into chunks of up to 512 tokens (use `tiktoken` for token counting) with 50-token overlap between consecutive chunks. When a section heading pattern is detected nearby (e.g., "Item 1A.", "Section 3:", numbered headings), prefer to break at the section boundary rather than the exact token count, even if that produces a slightly shorter or longer chunk. Any chunk whose source content came from a detected table (passed in via the `tables` parameter) should be marked `is_table=True` on the returned `Chunk` object. Write tests using PDF fixtures from `tests/fixtures/sample_filings/`, including at least one table-heavy document, asserting chunks don't split mid-sentence at arbitrary points and that table-derived chunks are correctly flagged.

AGENT-05 — Embedding & pgvector Indexing

Priority: Must-have · **Depends on:** AGENT-04

What to build: `rag/embeddings.py` — embeds each chunk using OpenAI's `text-embedding-3-small` and writes it to the `filing_chunks` table's `embedding` column.

Acceptance criteria: - [ ] Chunks are embedded in batches (not one API call per chunk) to control cost and latency - [ ] Every `filing_chunks` row for a processed filing has a non-null `embedding` value before the filing proceeds to the retrieval step - [ ] A failed embedding call retries with backoff; a filing doesn't silently proceed with partial embeddings - [ ] Cost per filing for this step is logged for the Langfuse cost dashboard (EVAL-06 dependency)

AI Coding Prompt: > Implement `src/regradar/rag/embeddings.py` with a function `embed_chunks(chunks: list[Chunk]) -> None` that batches the chunk texts (batches of up to 100) and calls OpenAI's embeddings endpoint (`text-embedding-3-small`) for each batch, then writes the resulting 1536-dimension vectors into the corresponding `filing_chunks.embedding` column via a bulk update. Retry a failed batch call up to twice with exponential backoff before raising, and ensure the function only reports success for a filing once every one of its chunks has a non-null embedding — never leave a filing partially embedded without an explicit error state. Emit a log line with the token count and estimated cost for each batch call so it can be picked up by cost-tracking (EVAL-06 depends on this data existing).

AGENT-06 — RAG Retrieval Agent (Hybrid Search + Rerank)

Priority: Must-have · **Depends on:** AGENT-05

What to build: The real `retrieve` graph node — given a new filing, finds the top-5 most similar past filings using hybrid BM25 + dense vector search via LlamaIndex, then reranks.

Acceptance criteria: - [ ] Combines a BM25 keyword search over `filing_chunks.chunk_text` with a pgvector cosine-similarity search over `embedding`, then reranks the combined candidate set - [ ] Returns the top 5 results by default (configurable), each including the source filing ID and the matched chunk text - [ ] If there's no prior history yet (empty corpus), returns an empty result set without error rather than failing the pipeline - [ ] Achieves Ragas context recall >0.80 against a labeled test set (validated in EVAL-01, but the retrieval quality itself is this ticket's responsibility)

AI Coding Prompt: > Implement `src/regradar/rag/retriever.py` and wire it into the `retrieve` node in `agents/rag_retrieval_agent.py`. Use LlamaIndex 0.10 to build a hybrid retriever combining BM25 keyword search over `filing_chunks.chunk_text` with a pgvector dense-vector similarity search over `filing_chunks.embedding`, and apply a reranking step over the combined candidate set (a cross-encoder reranker or LlamaIndex's built-in reranking node postprocessor). The retriever should return the top 5 most relevant past filings (configurable via a `top_k` parameter) relative to the current filing's content, excluding the current filing itself. If the corpus is empty (first filings ever processed), return an empty list rather than raising an exception, and make sure the pipeline continues without this context rather than failing. Write an integration test seeding a handful of fixture filings into the test database and asserting a query filing retrieves the expected most-similar fixture.

AGENT-07 — Analysis Agent (Structured Extraction)

Priority: Must-have · **Depends on:** AGENT-06

What to build: The real `analyze` graph node — calls GPT-4o with a structured-output JSON schema to extract obligations, deadlines, risk flags, affected products, and key entities.

Acceptance criteria: - [ ] Uses OpenAI's structured output / JSON mode with an enforced schema matching the `extractions` table's fields - [ ] Every extracted obligation includes a source citation (a reference back to the specific chunk/section it came from), per the Security document's hallucination-mitigation requirement - [ ] A malformed response triggers one retry with a stricter prompt before the filing is marked as needing review - [ ] Extraction F1 exceeds 0.82 against the labeled obligation test set (validated in EVAL-04, but extraction quality is this ticket's responsibility)

AI Coding Prompt: > Implement `src/regradar/agents/analysis_agent.py` as the real `analyze` node. Call OpenAI's `chat/completions` endpoint with `model="gpt-4o"` and re-

sponse_format={"type": "json_schema", ...} enforcing a schema with fields: obligations (list of objects with description and source_citation), deadlines (list of objects with description and date), risk_flags (list of strings), affected_products (list of strings), key_entities (list of strings), competitor_mentions (list of strings). Every obligation must include a source_citation referencing the chunk or section it was extracted from — reject and retry the response if any obligation is missing one. Include the retrieved similar-filings context from AGENT-06 in the prompt for grounding. On a malformed or schema-violating response, retry once with an added instruction emphasizing strict schema compliance; if it fails again, set the filing's status to needs_review rather than saving incomplete data. Write the result to the extractions table, including raw_model_response for debugging. Write a test with a mocked OpenAI client covering a well-formed response, a malformed response that succeeds on retry, and a malformed response that fails twice.

AGENT-08 — Summarization Agent (Persona Briefs)

Priority: Must-have · **Depends on:** AGENT-07

What to build: The real summarize graph node — generates the 3–5 sentence executive brief plus the three persona-tailored summaries (CCO, Analyst, Engineer) using the tiered model (HF bart-large-cnn or the LLM fallback).

Acceptance criteria: - [ ] Produces all four brief fields (executive_brief, cco_summary, analyst_summary, engineer_summary) and writes them to the briefs table - [ ] executive_brief is 3–5 sentences (validated by a length/sentence-count check, not just trusted from the model) - [ ] Each persona summary reflects that persona's stated needs from the PRD (CCO: board-level risk framing; Analyst: obligations/deadlines framing; Engineer: nothing filing-specific beyond confirming pipeline completion — this one can be the shortest) - [ ] ROUGE-L score against human-written reference briefs exceeds 0.45 on the eval test set (validated in EVAL-02, but summary quality is this ticket's responsibility)

AI Coding Prompt: > Implement src/regradar/agents/summarization_agent.py as the real summarize node. Using the extraction result from AGENT-07 and the model selected by the tiered router (AGENT-09), generate four outputs: executive_brief (a plain-English summary, enforced to 3–5 sentences via a post-generation sentence-count check with one regeneration attempt if it's out of range), cco_summary (board-level framing: what happened, why it matters, what risk level, in under 50 words), analyst_summary (obligations and deadlines framing, structured as a short bulleted list of extracted requirements), and engineer_summary (a one-line confirmation suitable for a webhook/API consumer, e.g., filing type, risk level, and a link reference — not a narrative summary). Write all four to the briefs table along with which model_used produced them. Write a test asserting executive_brief sentence count is always within 3–5 sentences even when the underlying model's raw output isn't.

AGENT-09 — Tiered Model Routing

Priority: Should-have · **Depends on:** AGENT-02, AGENT-08

What to build: llm_routing/tiered_router.py — a single function every LLM-calling agent uses to decide which model tier (GPT-4o vs. the cheaper Granite/HF fallback) to use for a given filing, based on its risk level.

Acceptance criteria: - [ ] Critical/High risk filings route to GPT-4o; Low/Medium route to the cheaper tier, per ADR-04 in the PRD - [ ] The risk threshold for tier selection is a configuration value, not hardcoded in any agent - [ ] If the primary model for a tier is unavailable (API error), the router falls back to the other tier's model rather than failing the filing entirely - [ ] A test confirms routing decisions for all four risk levels match the documented policy, and that a simulated OpenAI outage triggers fallback to Granite

AI Coding Prompt: > Implement src/regradar/llm_routing/tiered_router.py with a function select_model(risk_level: str, task: str) -> ModelChoice that returns which model to use

for a given pipeline task (`"summarization"` or `"analysis"`) based on the filing's `risk_level`. Use a configuration value `TIER_ROUTING_RISK_THRESHOLD` (default `"high"`) — any filing at or above this risk level uses `TIER_HIGH_MODEL` (GPT-4o); anything below uses `TIER_LOW_MODEL` (Granite-13B via Hugging Face). Wrap actual model invocation with a circuit-breaker-style fallback: if the selected tier's provider raises a connection or rate-limit error, automatically retry once against the other tier's model and log that a fallback occurred. Write tests confirming Low/Medium route to the cheap tier, High/Critical route to GPT-4o, and that a simulated OpenAI failure causes a summarization call to fall back to Granite rather than failing outright.

AGENT-10 — Delivery Agent

Priority: Must-have · **Depends on:** AGENT-08

What to build: The real `deliver` graph node — fans out to Slack, email, and every registered webhook for the filing's organization, writing a `deliveries` row for each attempt.

Acceptance criteria: - [ ] Fires exactly one Slack message, one email (if configured), and one webhook POST per active registered webhook, per filing - [ ] Every delivery attempt (successful or not) produces a `deliveries` row with the correct `channel`, `status`, and `response_code` - [ ] A delivery already logged as `sent` for a given filing+channel is never re-sent (idempotency, per the Security document's duplicate-alert protection) - [ ] Webhook payloads are HMAC-signed using the target webhook's own `hmac_secret` (never a shared global secret)

AI Coding Prompt: > Implement `src/regradar/agents/delivery_agent.py` as the real `deliver` node, using the `delivery/slack_client.py`, `delivery/sendgrid_client.py`, and `delivery/webhook_dispatcher.py` modules (build these as part of this ticket if they don't exist yet — see DELIV-01 and DELIV-02 for their detailed specs). Before sending anything, check the `deliveries` table for an existing `sent` row matching this `filing_id` and `channel` — if one exists, skip that channel entirely (this is the idempotency guarantee that prevents duplicate alerts on retry). For each channel that hasn't been sent yet: send the Slack message using the persona-appropriate brief text, send the SendGrid email if an email destination is configured, and POST to every active webhook registered for the organization, signing each payload with that specific webhook's `hmac_secret` (never a shared secret across webhooks). Write a `deliveries` row for every attempt, successful or failed, with the resulting `status` and `response_code`. Write a test confirming a filing that already has a `sent` Slack delivery row does not trigger a second Slack send on a re-run of this node.

Epic: REST API

API-01 — FastAPI Skeleton + Health Check

Priority: Must-have · **Depends on:** FOUND-02

What to build: The FastAPI app factory in `api/main.py` with a `/health` endpoint, structured logging, and CORS/middleware setup ready for the endpoints in later tickets.

Acceptance criteria: - [ ] `GET /health` returns `200` with a JSON body confirming database and Redis connectivity, or `503` if either is unreachable - [ ] OpenAPI docs are automatically available at `/docs` - [ ] Requests and responses are logged with a request ID for traceability

AI Coding Prompt: > In `src/regradar/api/main.py`, create a FastAPI app factory function `create_app() -> FastAPI`. Add a `GET /health` endpoint that checks database connectivity (a simple `SELECT 1`) and Redis connectivity (a `PING`), returning `200` with `{"status": "ok", "database": "ok", "redis": "ok"}` if both succeed, or `503` with details on whichever failed. Add middleware that generates a unique request ID per request, attaches it to the logging context, and includes it in every response header as `X-Request-ID`. Ensure OpenAPI docs are enabled at `/docs` with a proper title and version pulled from `pyproject.toml`.

API-02 — API Key Authentication

Priority: Must-have · **Depends on:** API-01, SEC-01

What to build: A FastAPI dependency (`api/deps.py`) that validates the `Authorization` header against the `api_keys` table and attaches the resolved role/organization to the request context.

Acceptance criteria: - [ ] A request with no key, an invalid key, or a revoked (`is_active = false`) key returns `401` with the standard error envelope, before touching any business logic - [ ] A valid key updates `api_keys.last_used_at` - [ ] The resolved role and organization are available to every route handler via dependency injection, so route logic never re-parses the header itself - [ ] Keys are compared using their hash, never in plaintext, and the comparison is constant-time to avoid timing attacks

AI Coding Prompt: > In `src/regradar/api/deps.py`, implement a FastAPI dependency `get_current_key(authorization: str = Header(...)) -> AuthenticatedKey` that extracts the bearer token from the `Authorization` header, hashes it (bcrypt or argon2, matching however keys are hashed at creation time), looks up a matching row in `api_keys` by hash using a constant-time comparison, and raises an `HTTPException(401)` with the error envelope `{"error": {"code": "invalid_api_key", "message": "...", "request_id": "..."}}` if no match is found or the matched key has `is_active = False`. On success, update `last_used_at` to now, and return an `AuthenticatedKey` object carrying the key's role and organization for use in route handlers via `Depends(get_current_key)`. Write tests for: missing header, malformed header, valid key, revoked key, and unknown key.

API-03 — Rate Limiting Middleware

Priority: Must-have · **Depends on:** API-02

What to build: Per-key rate limiting using `api_keys.rate_limit_per_minute`, backed by Redis, applied to every authenticated route.

Acceptance criteria: - [ ] Requests beyond a key's per-minute limit receive `429` with a `Retry-After` header, using the standard error envelope - [ ] The limit is read per-key from `api_keys.rate_limit_per_minute`, falling back to `API_RATE_LIMIT_PER_MINUTE_DEFAULT` if unset - [ ] One key exceeding its limit never affects another key's ability to make requests

AI Coding Prompt: > In `src/regradar/api/middleware/rate_limit.py`, implement a Redis-backed sliding-window or token-bucket rate limiter applied after authentication (API-02). For each authenticated request, check and increment a per-key counter in Redis keyed by `api_key_id` and the current minute window, using the limit from `AuthenticatedKey.rate_limit_per_minute` (falling back to `settings.API_RATE_LIMIT_PER_MINUTE_DEFAULT`). If the limit is exceeded, return `429` with the standard error envelope `{"error": {"code": "rate_limit_exceeded", ...}}` and a `Retry-After` header indicating seconds until the window resets. Write a test simulating one key making requests up to and past its limit, and confirming a second key is unaffected.

API-04 — GET /v1/filings

Priority: Must-have · **Depends on:** API-02, AGENT-01

What to build: The filings list endpoint with `domain`, `risk`, `since`, and pagination query parameters, respecting the Executive role's brief-level-only restriction from the Security document.

Acceptance criteria: - [ ] Supports `?domain=`, `?risk=`, `?since=`, and pagination (`page`, `page_size`) query parameters, all optional - [ ] Response matches the schema documented in the Technical Architecture Document's API contract - [ ] A caller with the Executive role never receives full extraction

detail in this response — only brief-level fields, per the role permission matrix - [ ] Response time stays under 500ms P99 with a realistically sized dataset (validated more thoroughly in LAUNCH-01)

AI Coding Prompt: > In `src/regradar/api/routers/filings.py`, implement `GET /v1/filings` using FastAPI and the `AuthenticatedKey` dependency from API-02. Accept optional query parameters `domain`, `risk`, `since` (ISO date), `page` (default 1), `page_size` (default 20, max 100). Query the `filings` table with these filters applied, joined with `briefs` for the executive_brief excerpt. Return a paginated response: `{"data": [...], "page": ..., "total": ...}`, where each item includes `id`, `entity_name`, `filing_type`, `domain`, `risk_level`, `published_at`, and `executive_brief`. If the authenticated key's role is `"executive"`, exclude any fields beyond this brief-level set even if requested — do not include raw extraction data in the response for that role under any query parameter combination. Write a test confirming an executive-role key never receives extraction fields, and that filtering by `domain` and `risk` narrows results correctly.

API-05 — GET /v1/filings/{id}

Priority: Must-have · **Depends on:** API-04

What to build: The single-filing detail endpoint returning the full record: brief, extraction, risk score, and similar past filings.

Acceptance criteria: - [ ] Returns `404` with the standard error envelope for a filing ID that doesn't exist or doesn't belong to the caller's organization - [ ] Includes extraction detail only for roles permitted to see it (Admin, Analyst, Legal Counsel, Eng Lead — not Executive), per the permission matrix - [ ] Includes a `similar_filings` array populated from the RAG retrieval results stored during processing

AI Coding Prompt: > In `src/regradar/api/routers/filings.py`, implement `GET /v1/filings/{filing_id}`. Look up the filing, its `briefs` row, its `extractions` row, and the top similar past filings from the retrieval step. Return `404` with the standard error envelope if the filing doesn't exist. Include `extractions` fields in the response only if the caller's role is Admin, Analyst, Legal Counsel, or Eng Lead — omit them entirely (not just null them out) for the Executive role. Include a `similar_filings` field: a list of `{id, entity_name, filing_type, published_at}` for the filings retrieved as context during this filing's processing. Write a test confirming the 404 case and the role-based field omission.

API-06 — POST /v1/filings/search (RAG Q&A)

Priority: Must-have · **Depends on:** AGENT-06, API-02

What to build: The natural-language search endpoint powering the Legal Counsel persona's "what has the SEC said about X" use case.

Acceptance criteria: - [ ] Accepts `{query: string, top_k: int}` and returns ranked filing matches with the matched excerpt and a generated natural-language answer synthesized from the results - [ ] Not accessible to the Executive role, per the permission matrix - [ ] Returns a clear, structured empty result (not an error) when no relevant filings exist - [ ] Falls back to keyword-only search with a note in the response if the answer-generation model is unavailable, per the Security document's error handling guide

AI Coding Prompt: > In `src/regradar/api/routers/filings.py`, implement `POST /v1/filings/search` accepting a JSON body `{"query": str, "top_k": int = 5}`. Use the retriever from `rag/retriever.py` (built in AGENT-06) to find the most relevant chunks/filings for the query, then generate a natural-language answer synthesizing those results using an LLM call, with citations back to the source filings. Return `{"answer": str, "sources": [{"filing_id": ..., "excerpt": ..., "entity_name": ...}]}`. Reject the request with `403` if the caller's role is `"executive"`. If the corpus has no relevant matches, return an empty `sources` array and an answer noting nothing relevant was found, rather than a 404 or error. If the answer-generation LLM call fails, fall back to returning just the keyword/vector search results with `"answer": null` and a `"degraded": true` flag rather than failing

the whole request. Write a test for the empty-results case and the degraded-fallback case using a mocked LLM failure.

API-07 — GET /v1/filings/{id}/brief

Priority: Must-have · **Depends on:** AGENT-08, API-05

What to build: The persona-specific brief endpoint, returning exactly one of the four brief variants based on the ?persona= query parameter.

Acceptance criteria: - [ ] ?persona=cco|analyst|engineer returns only that persona's summary text, not all four - [ ] Omitting ?persona= defaults to executive_brief - [ ] An invalid persona value returns 422 with the standard error envelope naming the valid options - [ ] A caller's role restricts which persona values they can request (e.g., an Executive-role key can only ever request cco, even if it tries another value)

AI Coding Prompt: > In src/regradar/api/routers/filings.py, implement GET /v1/filings/{filing_id}/brief. Default to returning executive_brief if persona is omitted. Return 422 with the standard error envelope if persona is provided but not one of the valid values. Enforce role restrictions: an Executive-role key's requests are always served the cco_summary regardless of the persona parameter value it passes, rather than erroring — silently narrowing to what that role is allowed to see, consistent with how GET /v1/filings already behaves for that role. Write a test covering each valid persona value, the default case, the invalid-value case, and the Executive-role override case.

API-08 — Webhook Registration & HMAC Signing

Priority: Must-have · **Depends on:** API-02, SEC-01

What to build: POST /v1/webhooks and DELETE /v1/webhooks/{id}, generating a per-webhook HMAC secret shown exactly once.

Acceptance criteria: - [ ] POST /v1/webhooks generates a unique hmac_secret per webhook, returns it once in the creation response, and never returns it again in any subsequent GET - [ ] A key can only see/manage webhooks it created, unless its role is Admin (per the row-level security rules) - [ ] DELETE /v1/webhooks/{id} returns 404 (not 403) for a webhook belonging to a different key, to avoid confirming that ID's existence to someone who shouldn't see it - [ ] Webhook URL is validated against the SSRF rule from SEC-02 before the row is created

AI Coding Prompt: > In src/regradar/api/routers/webhooks.py, implement POST /v1/webhooks accepting {"url": str, "filter_domain": str | None, "filter_min_risk": str | None}. Validate the URL using the SSRF-prevention helper from SEC-02 before proceeding — reject with 422 if it points at a private/internal address. Generate a cryptographically random hmac_secret (32+ bytes, base64-encoded), create the webhooks row associated with the calling key's api_key_id, and return the full webhook object including the plaintext hmac_secret in this one response only — no other endpoint should ever return this field again (exclude it from the Pydantic response model used elsewhere). Implement DELETE /v1/webhooks/{webhook_id}: if the webhook doesn't exist or belongs to a different api_key_id (and the caller isn't Admin-role), return 404, not 403, so the endpoint doesn't leak whether that ID exists to a caller who shouldn't know. Write tests for: successful creation showing the secret once, a follow-up GET confirming the secret isn't returned, and the ownership-based 404 behavior.

API-09 — GET /v1/metrics

Priority: Should-have · **Depends on:** EVAL-01, EVAL-06

What to build: The metrics endpoint surfacing eval scores, alert precision/recall, latency, and cost-per-filing from the `eval_runs` table, restricted to Admin and Eng Lead roles.

Acceptance criteria: - [ ] Returns the latest `eval_runs` row's metrics by default, with an optional date-range query for historical trend data - [ ] Returns 403 for any role other than Admin or Eng Lead, per the row-level security rule scoping this table - [ ] Response fields match `eval_runs`' documented targets (faithfulness >0.87, etc.) so a caller can compare current vs. target at a glance

AI Coding Prompt: > In `src/regradar/api/routers/metrics.py`, implement `GET /v1/metrics?since=&unti` restricted via role check to `admin` and `eng_lead` roles only (return 403 with the standard error envelope otherwise). Without date parameters, return the most recent `eval_runs` row. With date parameters, return a list of rows in that range for trend charting. Include the documented target value alongside each metric in the response (e.g., `{"ragas_faithfulness": {"value": 0.89, "target": 0.87}}`) so a consumer doesn't need to hardcode targets separately. Write a test confirming non-Admin/Eng-Lead roles are rejected.

API-10 — POST /v1/config/sources

Priority: Should-have · **Depends on:** API-02

What to build: Admin-only endpoint to update `source_configs` — which regulators/domains are monitored.

Acceptance criteria: - [ ] Restricted to Admin role; all other roles receive 403 - [ ] Validates that `sources` values are within the supported enum (SEC, FDA, FINRA) before writing - [ ] Changes take effect on the next scheduled ingestion cycle without requiring a redeploy

AI Coding Prompt: > In `src/regradar/api/routers/config.py`, implement `POST /v1/config/sources` accepting `{"sources": ["SEC", "FDA", "FINRA"], "domains": [...]}`, restricted to Admin-role keys only (403 otherwise). Validate that every value in `sources` is one of the supported enum values, returning 422 listing the invalid ones if not. Update or create the corresponding `source_configs` rows, setting `is_active = true` for included sources and `is_active = false` for any previously active source now omitted. Confirm via a test that the Prefect ingestion flow (ING-04) picks up this change on its next scheduled run without needing a restart, since it queries `source_configs` fresh each cycle.

API-11 — GET /v1/me (Session/Role Resolution)

Priority: Nice-to-have · **Depends on:** API-02, FE-02

What to build: A lightweight endpoint the future dashboard calls once on load to know the current session's role and organization, per the Frontend Specification Document's recommendation.

Acceptance criteria: - [ ] Returns `{role, organization_id, display_name}` for the authenticated session - [ ] Works with both the SSO-session-derived identity (once FE-02 exists) and a direct API key, so it's useful in both contexts - [ ] Returns 401 cleanly if called without a valid session/key

AI Coding Prompt: > In `src/regradar/api/routers/me.py`, implement `GET /v1/me` using the existing `get_current_key` dependency (API-02). Return `{"role": ..., "organization_id": ..., "display_name": ...}` describing the authenticated caller. This should work identically whether the caller authenticated via a direct API key or via a session cookie that resolves to an API key behind the scenes (once SSO/FE-02 exists) — the route logic itself shouldn't need to know which. Return 401 with the standard error envelope if authentication fails.

Epic: Security

SEC-01 — Row-Level Security Policies

Priority: Must-have · **Depends on:** FOUND-02

What to build: Postgres RLS policies for every customer-data table, implementing the rules from the Security & Access Document — deny by default, with explicit per-role, per-table allow rules.

Acceptance criteria: - [ ] RLS is enabled on `filings`, `filing_chunks`, `extractions`, `briefs`, `webhooks`, `deliveries`, `source_configs`, `eval_runs`, and `api_keys` - [ ] No customer-facing role can `INSERT`/`UPDATE`/`DELETE` on `filings`, `extractions`, or `briefs` — only the internal service role can - [ ] `webhooks` rows are only visible/manageable by their creating key, except for Admin - [ ] An automated test suite attempts, for every role, to access rows it should not be able to, and confirms each attempt is denied

AI Coding Prompt: > Write Alembic migrations enabling row-level security on all nine tables in the RegRadar schema (`filings`, `filing_chunks`, `extractions`, `briefs`, `webhooks`, `deliveries`, `source_configs`, `eval_runs`, `api_keys`), with `ALTER TABLE ... ENABLE ROW LEVEL SECURITY` and no default-allow policy (deny by default). Add policies matching these rules: any authenticated role can `SELECT` from `filings`, `extractions` (except Executive role — no `SELECT` for that role on `extractions`), and `briefs`; only a database role representing the internal service credential can `INSERT`/`UPDATE` on `filings`, `filing_chunks`, `extractions`, and `briefs`; `webhooks` rows are `SELECT`/`UPDATE`/`DELETE`-able only where `api_key_id` matches the current session's key, or where the current session's role is `admin`; `source_configs` is `SELECT`-able by all roles but only `INSERT`/`UPDATE`/`DELETE`-able by `admin`; `eval_runs` is `SELECT`-able only by `admin` and `eng_lead` roles. Use a Postgres session variable (e.g., set via `SET LOCAL app.current_role` and `app.current_key_id` at the start of each request's transaction) to make the current caller's identity available to policy expressions. Write a pytest suite that, for each role, attempts a forbidden read/write against each table and asserts it's rejected by the database itself, not just by application code.

SEC-02 — Webhook URL SSRF Validation

Priority: Must-have · **Depends on:** API-08

What to build: A validator that rejects webhook URLs pointing at internal/private network ranges before a webhook is ever registered or dispatched to.

Acceptance criteria: - [ ] Rejects URLs resolving to RFC 1918 private ranges, loopback, link-local, and cloud metadata endpoints (e.g., `169.254.169.254`) - [ ] Re-validates at dispatch time, not just at registration time (DNS can change between registration and delivery) - [ ] Rejects non-`https` URLs in production environments

AI Coding Prompt: > Implement `src/regradar/delivery/webhook_dispatcher.py`'s validation helper `validate_webhook_url(url: str) -> None`, raising a `ValueError` if the URL: is not `https://` (in non-development environments), or resolves (via actual DNS resolution, not just string inspection, to catch DNS-rebinding attempts) to a private IP range (RFC 1918: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), loopback (127.0.0.0/8), link-local (169.254.0.0/16, which also covers the AWS/GCP metadata endpoint), or any other non-public range. Call this validator both at webhook creation time (API-08) and immediately before every dispatch attempt in the delivery agent (AGENT-10), since DNS resolution can change between registration and send time. Write tests covering a public HTTPS URL (allowed), a private-IP URL, a metadata-endpoint URL, and a non-HTTPS URL.

SEC-03 — Webhook Replay Protection

Priority: Must-have · **Depends on:** API-08

What to build: Timestamp-based replay protection added to every signed webhook payload, so a captured, validly-signed request can't be resent later.

Acceptance criteria: - [ ] Every outbound webhook payload includes a timestamp, included in the HMAC signature computation (not just appended unsigned) - [ ] The dispatcher documentation (and a reference verification snippet for customers) shows rejecting any payload older than a defined window (recommend 5 minutes) - [ ] A test confirms two identical payloads sent at different timestamps produce different signatures

AI Coding Prompt: > Extend `src/regradar/delivery/webhook_dispatcher.py` so every outbound webhook request includes a `X-RegRadar-Timestamp` header (Unix timestamp) and computes the HMAC signature over the concatenation of the timestamp and the raw request body (not the body alone), following the same pattern Stripe uses for webhook signing. Document, in a docstring or `docs/api-contract.md`, the exact verification algorithm a receiving server should use, including rejecting any request whose timestamp is more than 5 minutes from the receiver's current time. Write a test confirming that signing the same payload at two different timestamps produces two different signature values, and that the verification helper (write one for use in tests/customer reference) correctly rejects a stale timestamp.

SEC-04 — Duplicate Filing Race Protection

Priority: Must-have · **Depends on:** ING-01

What to build: A database-level guarantee (not just application logic) that two near-simultaneous ingestion runs can never create two `filings` rows for the same source document.

Acceptance criteria: - [ ] A unique constraint exists on `(source, source_document_id)` in the `filings` table - [ ] A duplicate insert attempt fails at the database level and is caught and handled gracefully (logged, not crashed) by the ingestion code, rather than relying on an application-level "check then insert" that could race - [ ] A test simulates two concurrent insert attempts for the same document and confirms only one row exists afterward

AI Coding Prompt: > Add a database-level unique constraint on `(source, source_document_id)` in the `filings` table via an Alembic migration. Update every ingestion connector (`ING-01`, `ING-02`, `ING-03`) to catch the resulting integrity error on insert and treat it as "already exists, skip" rather than crashing the ingestion run. Write a test that fires two concurrent insert attempts (using threads or async tasks) for the same `(source, source_document_id)` pair and confirms exactly one row exists in the table afterward, with the second attempt handled gracefully rather than raising an unhandled exception.

SEC-05 — Organization-Scoping Scaffolding

Priority: Should-have · **Depends on:** SEC-01

What to build: An `organization_id` column added to every customer-data table now, even while only one organization exists, per the Security document's forward-compatibility recommendation.

Acceptance criteria: - [ ] `organization_id` exists on `filings`, `webhooks`, `deliveries`, `source_configs`, `api_keys` (and any table that logically belongs to one org) - [ ] All RLS policies from SEC-01 are updated to scope by `organization_id = current session's org`, not just role - [ ] A single default organization row exists for the current single-tenant deployment, and all existing/new data is associated with it

AI Coding Prompt: > Add an `organization_id` UUID column (foreign key to a new minimal `organizations` table with just `id` and `name`) to `filings`, `webhooks`, `deliveries`, `source_configs`, and `api_keys`, via an Alembic migration. Create a single default organization row and backfill `organization_id` on all existing rows to reference it. Update every RLS policy written in SEC-01 to additionally require `organization_id = current_setting('app.current_organization_id')::uuid`, so

the row-level security rules are scoped per-organization even though only one exists today. Update deps.py's get_current_key to also resolve and set the organization context per request. Write a test creating a second organization and confirming its data is fully isolated from the first under every existing RLS test from SEC-01.

Epic: Evaluation & Observability

EVAL-01 — Ragas Faithfulness + Context Recall Harness

Priority: Must-have · **Depends on:** AGENT-06, AGENT-07

What to build: eval/ragas_suite.py — runs Ragas's faithfulness and context recall metrics against a set of processed filings and writes results to eval_runs.

Acceptance criteria: - [ ] Computes faithfulness (are brief claims grounded in the source) and context recall (did retrieval find the relevant past filings), using the Ragas library - [ ] Runs against a fixed, version-controlled test set so results are comparable run-over-run - [ ] Writes a row to eval_runs with passed = true only if faithfulness >0.87 and context recall >0.80 - [ ] Can be invoked as a standalone script/CLI command, not just as part of CI, so it can be run ad hoc during development

AI Coding Prompt: > Implement src/regradar/eval/ragas_suite.py using the ragas library. Write a function run_ragas_eval(test_set_path: str) -> EvalResult that loads a fixed JSON test set (question/context/answer/ground_truth tuples derived from previously processed filings, stored under eval/fixtures/), runs Ragas's faithfulness and context_recall metrics against it, and returns the aggregate scores. Write the results to a new eval_runs row with run_type="manual" (or "pre_deploy_regression" when called from CI), setting passed=True only if ragas_faithfulness > 0.87 and ragas_context_recall > 0.80. Expose this as a CLI command (python -m regradar.cli run-eval ragas) so it can be run outside of CI during development. Commit an initial eval/fixtures/ragas_test_set.json with at least 20 labeled examples derived from fixture filings.

EVAL-02 — ROUGE-L Summarization Regression Suite

Priority: Must-have · **Depends on:** AGENT-08

What to build: eval/rouge_suite.py — compares generated executive_brief text against human-written reference briefs using ROUGE-L.

Acceptance criteria: - [ ] Uses Hugging Face's evaluate library's ROUGE implementation - [ ] Runs against a version-controlled set of filing/reference-brief pairs - [ ] Writes rouge_l to the same eval_runs row structure as EVAL-01, contributing to the same passed gate (>0.45 target)

AI Coding Prompt: > Implement src/regradar/eval/rouge_suite.py using Hugging Face's evaluate library's ROUGE metric. Write a function run_rouge_eval(test_set_path: str) -> float that loads a version-controlled set of {filing_id, generated_brief, reference_brief} triples from eval/fixtures/rouge_test_set.json, computes ROUGE-L between generated and reference briefs, and returns the average score. Integrate this into the same eval_runs row created by EVAL-01 for a given run (update rather than insert a second row for the same git_commit_sha/prompt_version), with passed requiring rouge_l > 0.45 in addition to the Ragas thresholds. Commit an initial eval/fixtures/rouge_test_set.json with at least 20 human-written reference briefs.

EVAL-03 — Alert Precision/Recall Test Set

Priority: Should-have · **Depends on:** AGENT-02

What to build: A 200-filing labeled test set and evaluator measuring how often a "Critical" alert turns out to be low-risk (false positive) and how often a genuinely Critical filing is missed (false negative).

Acceptance criteria: - [ ] Test set of 200 manually labeled filings exists under `eval/fixtures/` - [ ] Evaluator computes false-positive rate (<5% target) and false-negative rate (<3% target) against this set - [ ] Results are written to `eval_runs.alert_precision` / `alert_recall`

AI Coding Prompt: > Implement `src/regradar/eval/alert_precision_recall.py` with a function `run_alert_eval(test_set_path: str) -> tuple[float, float]` that runs the Triage Agent (AGENT-02) against a 200-filing labeled test set (`eval/fixtures/alert_test_set.json`, containing filing text and a human-assigned ground-truth risk level), and computes: false-positive rate (predicted Critical/High but actually Low/Medium) and false-negative rate (predicted Low/Medium but actually Critical/High). Write both to the current `eval_runs` row. Flag a note in the code if the committed fixture set has fewer than 200 examples, since the full set requires manual labeling effort likely completed outside of this ticket — set up the harness and pipeline against whatever subset exists initially, sized to grow.

EVAL-04 — Extraction F1 Eval

Priority: Should-have · **Depends on:** AGENT-07

What to build: Precision/recall/F1 scoring of the Analysis Agent's extracted obligations and deadlines against a labeled set.

Acceptance criteria: - [ ] Compares extracted obligations/deadlines against a labeled ground-truth set using a reasonable matching strategy (exact or fuzzy string match with a defined threshold) - [ ] Computes and writes `extraction_f1` to `eval_runs`, targeting >0.82

AI Coding Prompt: > Implement `src/regradar/eval/extraction_f1.py` with a function `run_extraction_eval(test_set_path: str) -> float` that runs the Analysis Agent (AGENT-07) against a labeled test set of filings with ground-truth obligations/deadlines (`eval/fixtures/extraction_test_se` matches predicted obligations to ground-truth obligations using fuzzy string similarity above a defined threshold (e.g., token-set ratio > 0.8 counts as a match), and computes precision, recall, and F1. Write `extraction_f1` to the current `eval_runs` row, targeting >0.82.

EVAL-05 — LangSmith Prompt Versioning Integration

Priority: Must-have · **Depends on:** AGENT-01

What to build: Every LLM-calling prompt in the pipeline (triage, analysis, summarization) is registered and versioned in LangSmith, with the active version recorded on each `eval_runs` row.

Acceptance criteria: - [ ] Every prompt template used by any agent is pulled from a LangSmith-managed prompt store, not hardcoded inline in agent code - [ ] Changing a prompt creates a new version, traceable in LangSmith's UI, without a code deploy for prompt-only changes - [ ] `eval_runs.prompt_version` accurately reflects which prompt version was active for that run

AI Coding Prompt: > Implement `src/regradar/observability/langsmith_config.py` to initialize LangSmith tracing for the project (using `LANGSMITH_API_KEY` and `LANGSMITH_PROJECT` from settings). Refactor the prompt templates currently used in `agents/triage_agent.py`, `agents/analysis_agent.py`, and `agents/summarization_agent.py` to be pulled from LangSmith's prompt hub rather than defined as inline strings, so prompt iteration doesn't require a code change. Ensure every LLM call made anywhere in the agent pipeline is automatically traced to LangSmith (via the standard LangChain/LangGraph LangSmith integration). When an eval run (EVAL-01/EVAL-02) executes, record the currently active prompt version identifier into that run's `eval_runs.prompt_version` field.

EVAL-06 — Langfuse Cost/Latency Dashboard & Budget Circuit Breaker

Priority: Must-have · **Depends on:** AGENT-09

What to build: Every LLM call is logged to Langfuse with token counts and cost; a running daily total triggers the tiered-fallback circuit breaker at `DAILY_COST_ALERT_THRESHOLD_USD`.

Acceptance criteria: - [ ] Every OpenAI and Hugging Face call anywhere in the pipeline logs cost and latency to Langfuse - [ ] A running daily cost total is tracked (Redis counter or equivalent), and crossing `DAILY_COST_ALERT_THRESHOLD_USD` triggers an alert to the engineering team - [ ] Beyond the alert, remaining filings that day are automatically routed to the cheaper model tier regardless of risk level, per PRD risk R4's mitigation, until the counter resets - [ ] `eval_runs.avg_cost_per_filing_usd` is computed from this logged data

AI Coding Prompt: > Implement `src/regradar/observability/langfuse_config.py` to wrap every OpenAI and Hugging Face call made anywhere in the agent pipeline with Langfuse tracing, logging token counts, latency, and computed cost per call. Implement a daily running-cost counter in Redis (keyed by date, incremented after every LLM call with that call's cost), and when the counter crosses `DAILY_COST_ALERT_THRESHOLD_USD`: (1) send an alert (log at ERROR level, and stub a notification hook for Slack/email alerting to the engineering team), and (2) set a Redis flag that the tiered router (AGENT-09) checks and, if set, forces all subsequent filings that day to the cheap tier regardless of risk level, until the flag resets at midnight UTC. Add a function `compute_avg_cost_per_filing(since: datetime) -> float` used to populate `eval_runs.avg_cost_per_filing_usd`. Write a test simulating cost accumulation crossing the threshold and confirming the tiered router respects the forced-cheap-tier flag afterward.

EVAL-07 — CI Eval Gate

Priority: Must-have · **Depends on:** EVAL-01, EVAL-02, FOUND-04

What to build: A GitHub Actions workflow, separate from the general CI pipeline, that runs the full eval suite on every prompt or agent-logic change and blocks the deploy if any threshold fails.

Acceptance criteria: - [ ] Triggers on changes to any file under `agents/`, `rag/`, `eval/`, or prompt-related files - [ ] Runs EVAL-01, EVAL-02, EVAL-03, and EVAL-04 and fails the workflow if any `eval_runs.passed` condition is false - [ ] The deploy workflow (`deploy.yml`) requires this workflow to have passed on the current commit before running

AI Coding Prompt: > Create `.github/workflows/eval-regression.yml`, triggered on pull requests that modify files under `src/regradar/agents/`, `src/regradar/rag/`, `src/regradar/eval/`, or any prompt-related file. The workflow should install the project, run the full eval suite (`ragas_suite.py`, `rouge_suite.py`, `alert_precision_recall.py`, `extraction_f1.py`), and fail the workflow (non-zero exit) if the resulting `eval_runs.passed` is `False` or any individual metric misses its documented target. Update `.github/workflows/deploy.yml` (create if it doesn't exist, per the deployment ticket scope) to require this eval-regression workflow to have succeeded on the current commit as a prerequisite before the deploy job runs, using GitHub Actions' `needs:` or required-status-check mechanism.

Epic: Delivery

DELIV-01 — Slack Delivery Integration

Priority: Must-have · **Depends on:** AGENT-10

What to build: `delivery/slack_client.py` — sends formatted Block Kit messages to the configured Slack Incoming Webhook.

Acceptance criteria: - [ ] Messages include entity name, filing type, risk level (color-matched, if using Block Kit formatting), and a link back to the filing - [ ] A non-200 response from Slack is treated as a

failed delivery and recorded as such in `deliveries` - [ ] Slack webhook URL is stored per-organization (via `source_configs` or an org settings table), not hardcoded

AI Coding Prompt: > Implement `src/regradar/delivery/slack_client.py` with a function `send_slack_alert(webhook_url: str, filing: Filing, brief: Brief) -> DeliveryResult` that POSTs a Slack Block Kit formatted message to `webhook_url`, including the entity name, filing type, a risk-level indicator, the `cco_summary` text, and a link back to the filing in RegRadar. Treat any non-`200` response, or a response body other than `"ok"`, as a failure. Return a `DeliveryResult` with `status` and `response_code` for the caller (AGENT-10) to write into the `deliveries` table. Write a test with a mocked HTTP client covering a successful send and a failed send (both a non-200 status and a network timeout).

DELIV-02 — SendGrid Email Integration

Priority: Must-have · **Depends on:** AGENT-10

What to build: `delivery/sendgrid_client.py` — sends the persona-appropriate brief as a formatted HTML email.

Acceptance criteria: - [ ] Uses SendGrid's `mail/send` API with a proper HTML template incorporating the brief text and risk level - [ ] A `202` response is treated as success; anything else as failure, recorded in `deliveries` - [ ] From-address and reply-to are configured via settings, not hardcoded

AI Coding Prompt: > Implement `src/regradar/delivery/sendgrid_client.py` with a function `send_email_alert(recipient: str, filing: Filing, brief: Brief) -> DeliveryResult` that calls SendGrid's `POST https://api.sendgrid.com/v3/mail/send` endpoint with a simple HTML template rendering the entity name, filing type, risk level, and the appropriate persona brief text, using `SENDGRID_FROM_EMAIL` from settings as the sender. Treat a `202` response as success and anything else as failure. Return a `DeliveryResult` for AGENT-10 to record. Write a test with a mocked HTTP client for both the success and failure paths.

DELIV-03 — Weekly Digest Job

Priority: Should-have · **Depends on:** DELIV-01, DELIV-02

What to build: A scheduled Celery task that compiles all Critical/High filings from the trailing 7 days into a single digest, delivered via email (and optionally Slack), per PRD user story US-07.

Acceptance criteria: - [ ] Runs on a weekly schedule (configurable day/time) - [ ] Queries `deliveries/filings` for the trailing 7 days' Critical/High filings, grouped by organization - [ ] Produces one digest email per organization, not one per filing - [ ] A week with zero Critical/High filings still sends a short "nothing to report" digest rather than silently skipping (so recipients trust the absence of alerts means an actually quiet week, not a broken digest)

AI Coding Prompt: > Implement `src/regradar/workers/digest_tasks.py` with a Celery periodic task `send_weekly_digest()`, scheduled via Celery Beat to run weekly (configurable time). For each organization, query all Critical/High filings from the trailing 7 days, and compose a single digest email (using `delivery/sendgrid_client.py`) listing each filing's entity name, risk level, and executive brief, grouped and sorted by risk level descending. If an organization has zero Critical/High filings that week, still send a short digest explicitly stating "No Critical or High risk filings this week" rather than skipping that organization silently. Write a test covering both the normal case (filings present) and the zero-filings case.

DELIV-04 — Delivery Failure Fallback

Priority: Should-have · **Depends on:** DELIV-01, DELIV-02

What to build: If Slack delivery fails for an organization, automatically fall back to email so a Critical alert is never silently lost, per the Security document's error handling guide.

Acceptance criteria: - [ ] A failed Slack delivery for a Critical/High filing automatically triggers an email send, even if email wasn't the primary configured channel - [ ] The fallback delivery is recorded as its own `deliveries` row, distinguishable from a normally-configured email delivery (e.g., a `note` or `is_fallback` flag) - [ ] Repeated Slack failures for the same organization trigger an alert to the account owner recommending reconnection (can be a logged event/stub notification for V1)

AI Coding Prompt: > Extend `src/regradar/agents/delivery_agent.py` (AGENT-10) so that when a Slack delivery attempt for a Critical or High risk filing fails, it automatically triggers an email send via `delivery/sendgrid_client.py` as a fallback, even for organizations that don't have email configured as a primary channel (fall back to a designated admin contact email if no other is configured). Record this fallback delivery as its own `deliveries` row with an added boolean field `is_fallback` (add via migration if not already present) so it's distinguishable in delivery logs from a normal email send. Track consecutive Slack failures per organization (a simple counter, e.g., in Redis) and, after 3 consecutive failures, log a distinct "Slack reconnection needed" event (stub the actual notification channel — this is a hook for a future admin-alerting mechanism). Write a test confirming a failed Slack send for a Critical filing results in exactly one fallback email delivery row.

Epic: Frontend Dashboard (V2)

These tickets build the dashboard specified in the Frontend Specification Document. Per the PRD, V1 ships API-first with no UI (NG3) — everything in this epic is explicitly deferred scope, included here so it's planned rather than forgotten.

FE-01 — Design System / Tailwind Token Implementation

Priority: Nice-to-have · **Depends on:** None

What to build: A Tailwind config and base component library implementing every token from the Frontend Specification Document's design system (colors, typography, spacing, radius, shadows) as reusable code, not just documentation.

Acceptance criteria: - [ ] `tailwind.config.ts` defines every color, spacing, radius, and shadow token from the Frontend Specification Document by name (e.g., `primary-600`, `risk-critical`) so they're usable as literal class names - [ ] Base components (Button, Input, Card, Modal, Badge, Table) exist as reusable React components matching the documented variants/states exactly - [ ] A Storybook (or equivalent) instance demonstrates every component variant and state

AI Coding Prompt: > Set up a React + TypeScript + Vite project with Tailwind CSS. In `tailwind.config.ts`, define custom color tokens matching these exact hex values: primary scale (`primary-50: #EEF2FF` through `primary-900: #312E81`, with `primary-600: #4F46E5` as the default brand color), slate neutral scale (`slate-50: #F8FAFC` through `slate-900: #0F172A`), risk-level colors (`risk-low: #16A34A`, `risk-medium: #D97706`, `risk-high: #EA580C`, `risk-critical: #DC2626`, each with a matching `-bg` light tint variant), and domain tag colors (`domain-financial: #2563EB`, `domain-clinical: #7C3AED`, `domain-environmental: #0D9488`). Set the font family to Inter with IBM Plex Mono for monospace. Define spacing tokens on the 4px base scale (4/8/12/16/24/32/48/64) and radius tokens (sm:6px, md:8px, lg:12px, xl:16px, full:9999px). Build reusable components in `src/components/`: `Button` (primary/secondary/destructive/ghost variants, sm/md/lg sizes, hover/focus/disabled/loading states), `Input` (default/focus/error/disabled states with label and helper text support), `Card`, `Modal`, `Badge` (risk and domain variants), and `Table` (with the 56px row height, sticky header, and hover-row spec). Set up Storybook with a story for every component variant and state.

FE-02 — SSO Login & Session Integration

Priority: Nice-to-have · **Depends on:** API-02

What to build: OIDC-based SSO login flow (Google/Microsoft) that establishes the `HttpOnly` session cookie described in the Frontend Specification Document, and calls `GET /v1/me` to drive role-aware UI.

Acceptance criteria: - [ ] A user can log in via Google or Microsoft OIDC and land on the Filings List with a valid session - [ ] The session is an `HttpOnly`, `Secure` cookie — never accessible to frontend JavaScript - [ ] `GET /v1/me` is called once on load and its role/organization used to hide navigation items the current role can't use

AI Coding Prompt: > Implement an OIDC-based SSO login flow for the RegRadar dashboard, supporting Google and Microsoft as identity providers. On successful login, the backend should set an `HttpOnly`, `Secure`, `SameSite=Lax` session cookie (implement the backend session-issuing endpoint as part of this ticket if it doesn't exist, translating the SSO identity into an underlying API key per the Security & Access Document's model). On the frontend, call `GET /v1/me` once on app load using this cookie, store the returned role/organization in a top-level auth context, and use it to conditionally render navigation items per the role permission matrix from the Security & Access Document (e.g., hide "Source Configuration" for non-Admin roles). Handle a `401` from any API call by redirecting to the login screen.

FE-03 — Filings List Screen

Priority: Nice-to-have · **Depends on:** FE-01, FE-02, API-04

What to build: The default landing screen — a filterable, paginated data table of filings using the Table component from FE-01 and data from `GET /v1/filings`.

Acceptance criteria: - [ ] Filters by domain, risk level, source, and date range, reflected in the URL query string (so a filtered view is shareable/bookmarkable) - [ ] Risk and domain badges use the exact colors from the design system - [ ] Loading, empty, and error states are implemented per the Table component spec - [ ] Clicking a row navigates to the Filing Detail screen (FE-04)

AI Coding Prompt: > Build the Filings List screen at `src/pages/FilingsList.tsx`, using the `Table` component from FE-01 and TanStack Query to fetch from `GET /v1/filings`. Implement filter controls for domain, risk level, source, and date range, syncing their state to the URL query string so a filtered view is bookmarkable/shareable. Render risk level and domain as `Badge` components using the exact color tokens from the design system. Implement loading (skeleton rows), empty (centered message + icon per the Table component's empty-state spec), and error (banner with retry action) states. Clicking a row navigates to `/filings/:id`. Use TanStack Query's stale-while-revalidate defaults so returning to this screen from Filing Detail feels instant.

FE-04 — Filing Detail Screen

Priority: Nice-to-have · **Depends on:** FE-03, API-05, API-07

What to build: The full filing view — persona-tabbed briefs, obligations/deadlines list, risk score, similar filings, link to original PDF, and a live-updating status via Supabase Realtime.

Acceptance criteria: - [ ] Tabs switch between persona briefs, calling `GET /v1/filings/{id}/brief?persona=` per tab - [ ] Obligations and deadlines render as a structured list, not raw JSON - [ ] Subscribes to Supabase Realtime for this filing's `status` field and updates the UI live without a manual refresh - [ ] "View Original Document" opens the pre-signed S3 URL from the backend in a new tab

AI Coding Prompt: > Build the Filing Detail screen at `src/pages/FilingDetail.tsx`. Fetch the filing via `GET /v1/filings/{id}` (API-05) and render: entity name, filing type, risk badge, domain tag, and a

tabbed section for the four persona briefs (fetching each via GET /v1/filings/{id}/brief?persona=... from API-07 as the user switches tabs, cached per persona). Render obligations and deadlines from the extraction data as a structured, readable list — not raw JSON — with each obligation showing its source citation. Show a similar_filings section as a list of linked cards. Add a "View Original Document" button that calls a backend endpoint for a pre-signed S3 URL and opens it in a new tab. Subscribe to Supabase Realtime on the filings table filtered to this filing's ID (per the Frontend Specification Document's real-time pattern) so the status indicator updates live from "processing" to "complete" without a manual refresh.

FE-05 — Ask RegRadar (RAG Search) Screen

Priority: Nice-to-have · **Depends on:** FE-01, API-06

What to build: The Legal Counsel persona's natural-language search screen, calling POST /v1/filings/search.

Acceptance criteria: - [ ] Uses the large search-input variant from the design system - [ ] Displays the synthesized answer with clickable citations linking to source filings - [ ] Handles the "degraded" response flag from API-06 by showing a note that natural-language answering is temporarily limited, still showing raw results - [ ] Not shown in navigation at all for the Executive role, per the permission model

AI Coding Prompt: > Build the Ask RegRadar screen at src/pages/Search.tsx, using the large (56px-height, 12px-radius) search input variant from the design system. On submit, call POST /v1/filings/search with the query. Render the returned answer text with inline citation markers linking to each source filing in sources. If the response includes "degraded": true, show a banner noting natural-language answering is temporarily limited, and still render the raw sources list. Hide this screen entirely from the sidebar navigation (not just block access to it) when the current session's role is executive, per FE-02's role-aware navigation.

FE-06 — Webhooks Management Screen

Priority: Nice-to-have · **Depends on:** FE-01, API-08

What to build: List, create, and delete webhooks, with the create flow showing the HMAC secret exactly once.

Acceptance criteria: - [ ] Create flow uses the Modal component, and displays the returned hmac_secret in a copyable field with an explicit "this won't be shown again" warning - [ ] Delete uses the destructive-confirm Modal pattern from the design system - [ ] List shows delivery success/failure indicators per webhook, sourced from recent deliveries rows

AI Coding Prompt: > Build the Webhooks screen at src/pages/Webhooks.tsx. List existing webhooks (URL, active status, filter settings, and a delivery-health indicator derived from recent delivery success/failure). Implement a "Register Webhook" flow using the Modal component from FE-01: on successful creation via POST /v1/webhooks, show the returned hmac_secret once in a copyable code field with a clear, non-dismissable-without-acknowledgment warning that it will never be shown again. Implement delete using the destructive-confirm Modal pattern (red Confirm button, clear consequence statement) calling DELETE /v1/webhooks/{id}.

FE-07 — API Keys Management Screen

Priority: Nice-to-have · **Depends on:** FE-01, FE-02

What to build: List, create, and revoke API keys (Admin-only), following the same show-once pattern as webhooks.

Acceptance criteria: - [ ] Only accessible/visible in navigation for Admin role - [ ] New key's plaintext value is shown exactly once, in a copyable field, with a clear warning - [ ] List shows masked key identifiers, role, `last_used_at`, and active status — never the plaintext value - [ ] Revoke uses the destructive-confirm Modal pattern

AI Coding Prompt: > Build the API Keys screen at `src/pages/ApiKeys.tsx`, visible only to Admin-role sessions. List existing keys showing a masked identifier (e.g., last 4 characters), assigned role, `last_used_at`, and active status — the plaintext key is never available here. Implement a "Create Key" flow (calling whichever backend endpoint issues new keys — build this endpoint under `api/routers/` as part of this ticket if it doesn't already exist, restricted to Admin role, following the same hash-and-show-once pattern as webhooks in API-08) that displays the new key's plaintext value exactly once in a copyable field with a clear warning it won't be shown again. Implement revoke using the destructive-confirm Modal pattern.

FE-08 — Metrics & Cost Dashboard Screen

Priority: Nice-to-have · **Depends on:** FE-01, API-09

What to build: Admin/Eng-Lead-only dashboard of the metric cards and trend charts specified in the Frontend Specification Document, sourced from `GET /v1/metrics`.

Acceptance criteria: - [ ] Metric cards show current value, target, and a trend indicator (colored correctly per whether the direction is an improvement or regression for that specific metric) - [ ] A trend chart shows faithfulness/cost-per-filing over time using the date-range variant of `GET /v1/metrics` - [ ] Not visible in navigation for any role other than Admin/Eng Lead

AI Coding Prompt: > Build the Metrics & Cost screen at `src/pages/Metrics.tsx`, visible only to Admin/Eng-Lead-role sessions. Fetch current metrics via `GET /v1/metrics` and render each as a `MetricCard` component (per the Frontend Specification Document's spec: label, large value, trend arrow) for faithfulness, context recall, ROUGE-L, alert precision/recall, P99 latency, and cost-per-filing — each card's trend arrow should be colored green when the metric is moving in the desirable direction (e.g., rising faithfulness, falling cost) and red otherwise. Add a trend chart (using a lightweight charting library) plotting cost-per-filing and faithfulness over the trailing 30 days, sourced from `GET /v1/metrics?since=&until=`.

FE-09 — Source Configuration Screen

Priority: Nice-to-have · **Depends on:** FE-01, API-10

What to build: Admin-only screen with toggles for which regulators/domains are monitored, calling `POST /v1/config/sources`.

Acceptance criteria: - [ ] Toggles for SEC/FDA/FINRA and each domain, reflecting current `source_configs` state on load - [ ] Changes require an explicit "Save" action (not auto-saved on every toggle), consistent with the design system's "no optimistic UI for consequential actions" rule - [ ] Confirms success or shows the specific validation error from the backend if the save fails

AI Coding Prompt: > Build the Source Configuration screen at `src/pages/SourceConfig.tsx`, visible only to Admin-role sessions. Show toggle controls for each regulator (SEC, FDA, FINRA) and each domain (financial, clinical, environmental, other), initialized from the current backend state (fetch via whichever GET endpoint exposes `source_configs` — add one under `api/routers/config.py` as part of this ticket if `POST /v1/config/sources` doesn't already have a matching GET). Changes are staged locally and only submitted on an explicit "Save Changes" button click — do not auto-save on toggle, consistent with the design system's rule against optimistic UI for consequential actions. On save, call `POST /v1/config/sources`; show a success toast on `200`, or render the specific validation error inline if the backend returns 422.

Epic: Launch Readiness

LAUNCH-01 — Load Testing (100 Concurrent Filings)

Priority: Must-have · **Depends on:** API-04 through API-08, DELIV-01, DELIV-02

What to build: A load test simulating 100 filings being processed concurrently through the full pipeline, validating the <3 min filing-to-alert latency and <500ms API P99 targets under load.

Acceptance criteria: - [ ] Simulates 100 filings entering the pipeline within a short window and measures end-to-end time to Slack/email delivery for each - [ ] Simultaneously load-tests the read API endpoints (`/v1/filings`, `/v1/filings/{id}`, `/v1/filings/search`) to confirm P99 stays under 500ms while the pipeline is under load - [ ] Produces a report documenting actual latency/throughput numbers, committed to `docs/` for reference - [ ] Any bottleneck found is either fixed or explicitly documented as a known limitation (feeds LAUNCH-02)

AI Coding Prompt: > Write a load test (using `locust` or a custom `asyncio`-based script) under `tests/load/` that submits 100 fixture filings into the pipeline concurrently (via direct Celery task enqueue, simulating what the ingestion agent would produce) and measures, for each, the elapsed time from enqueue to a `sent` row appearing in `deliveries`. Simultaneously, run a separate load generator hitting `GET /v1/filings`, `GET /v1/filings/{id}`, and `POST /v1/filings/search` at a realistic request rate throughout the test window, measuring P99 response time. Produce a markdown report at `docs/load-test-results.md` summarizing: end-to-end filing-to-alert latency distribution (target: <3 min), API P99 latency during load (target: <500ms), and any errors or timeouts observed. Flag any target miss clearly rather than omitting it.

LAUNCH-02 — Known-Limitations Documentation

Priority: Should-have · **Depends on:** EVAL-01 through EVAL-04

What to build: A written, honest account of the system's failure modes — hallucination rate by document type, recall gaps on tabular data, latency behavior under load — based on real eval results, not aspirational targets.

Acceptance criteria: - [ ] Documents actual measured hallucination rate, broken down by filing type/domain if the data supports it - [ ] Documents actual recall performance specifically on table-heavy filings vs. prose-heavy filings - [ ] Documents actual load-test results from LAUNCH-01 - [ ] Lives in `docs/` as a committed, version-controlled document, not a one-off Slack message or forgotten wiki page

AI Coding Prompt: > Write `docs/known-limitations.md` summarizing RegRadar's actual measured limitations, sourced from the eval harness results (EVAL-01 through EVAL-04) and the load test (LAUNCH-01) rather than aspirational targets. Include: measured hallucination rate overall and, if the eval fixture set supports the breakdown, by filing type or domain; measured extraction/retrieval recall specifically on table-heavy vs. prose-heavy filings (cross-reference the `is_table` flag on `filing_chunks`); actual observed latency and any bottlenecks found during load testing; and any known gaps in ingestion coverage (e.g., FINRA if ING-03 isn't complete yet, OCR for scanned PDFs since LAUNCH-03 is out of scope). Write this as an honest, specific document — vague statements like "performance may vary" should be replaced with actual numbers wherever the eval data provides them.

LAUNCH-03 — OCR Fallback for Scanned PDFs

Priority: Nice-to-have · **Depends on:** AGENT-04

What to build: Explicitly out of V1 scope per the PRD's cut list — included here so it's planned for V1.1/V2 rather than forgotten, given it's flagged as a known limitation in LAUNCH-02.

Acceptance criteria: - [ ] Scanned (image-only) PDFs are detected and routed through OCR (e.g., `pytesseract`) before chunking, rather than producing an empty or garbled extraction - [ ] OCR'd text is flagged in `filing_chunks` (e.g., a `source_ocr` boolean) so downstream quality metrics can be tracked separately for OCR'd vs. native-text filings

AI Coding Prompt: > Extend `src/regradar/rag/chunking.py` (built in AGENT-04) to detect when a filing's PDF produces no or near-zero extractable text (a strong signal it's a scanned image), and in that case run it through OCR using `pytesseract` (converting pages to images via `pdf2image` first) before chunking proceeds as normal. Add a `source_ocr` boolean field to `filing_chunks` (via migration) set to `true` for any chunk derived from OCR'd text, so downstream eval metrics (EVAL-01 through EVAL-04) can be broken out separately for OCR'd vs. native-text filings. Write a test using a scanned-PDF fixture confirming OCR is triggered and text is successfully extracted.

LAUNCH-04 — Architecture Decision Records for Build-Time Choices

Priority: Should-have · **Depends on:** None (ongoing)

What to build: New ADRs, following the format of the four already written in the PRD, documenting any additional significant technical decisions made during implementation that aren't already covered.

Acceptance criteria: - [ ] At minimum, ADRs exist for: the row-level security implementation approach (SEC-01), the tiered model routing thresholds actually chosen (AGENT-09), and the frontend stack choice (FE-01) if the dashboard is built - [ ] Each ADR follows the existing format: Decision, Rationale, Tradeoff - [ ] ADRs are written at the time the decision is made, not reconstructed after the fact from memory

AI Coding Prompt: > Following the exact format of `ADR-01` through `ADR-04` already in the RegRadar PRD (Decision / Rationale / Tradeoff), write new ADRs under `docs/adrs/` for any significant technical decision made during implementation that isn't already covered by the existing four — at minimum, one for the row-level security implementation approach chosen in SEC-01 (e.g., session-variable-based policies vs. an alternative), one for the specific confidence threshold and fallback behavior chosen for tiered model routing in AGENT-09, and one for the frontend stack decision if FE-01 is built. Number them sequentially continuing from `ADR-04` (i.e., `ADR-05`, `ADR-06`, …). Keep each to the same length and style as the existing four — a few sentences per section, not an essay.